Decentralized ML Hub: A Native Edge-Based Federated Learning Platform

Naman Limani

***Abstract*—As machine learning models increasingly rely on vast amounts of data, privacy and data security have become paramount concerns. Federated Learning (FL) offers a paradigm shift by bringing the model to the data rather than centralizing sensitive datasets. This paper presents the Decentralized ML Hub, an end-to-end Federated Learning platform designed for seamless deployment across edge devices. By utilizing a native, compiled edge-client application, the architecture entirely bypasses the need for end-user Docker installations or terminal usage. The system integrates a centralized cloud aggregation server with distributed local training environments, utilizing RESTful APIs to navigate strict cloud firewalls. This report outlines the system's objective, core architecture, comprehensive UI design, deployment challenges, and current limitations.**

***Index Terms*—Federated Learning, Edge Computing, PyTorch, Electron, Distributed Systems**

I. Introduction & Objective

Traditional machine learning infrastructure requires organizations to upload their highly sensitive, proprietary data to a centralized cloud server for model training. This poses severe security risks and regulatory compliance challenges.

The primary objective of the Decentralized ML Hub is to provide a robust, privacy-preserving infrastructure where a central Artificial Intelligence (AI) model can be collaboratively trained across multiple remote nodes. Instead of transmitting raw data, the global model is distributed to the edge nodes. These nodes train the model natively using local hardware and private data, subsequently transmitting only the updated mathematical weights back to the central hub for aggregation.

II. System Architecture & Working

The architecture is divided into two primary environments: the Cloud Hub and the Edge Node Client.

A. The Cloud Hub (Central Aggregator)

The backend acts as the central orchestrator, built using FastAPI (Python). It is supported by a Serverless PostgreSQL database (Neon) for user/job state management and a Serverless Redis broker (Upstash) for queuing tasks. A Celery worker asynchronously handles the computationally heavy aggregation of weights received from edge nodes to formulate the new global model.

B. The Edge Node Client (Trojan Horse Architecture)

To ensure zero-friction onboarding for non-technical users, the edge client avoids Docker entirely. The client consists of an Electron.js desktop wrapper encapsulating a fully compiled PyInstaller PyTorch executable. When a user initiates training, the React UI dynamically passes absolute file paths to the hidden native Python engine. The engine natively reads the local dataset, trains the Convolutional Neural Network (CNN) or Multi-Layer Perceptron (MLP), evaluates local accuracy, and streams the updated weights back to the cloud.

III. User Interface Design: Admin Portal

The Admin Web UI provides centralized oversight of the federated network, allowing project managers to monitor global learning progress and node participation.

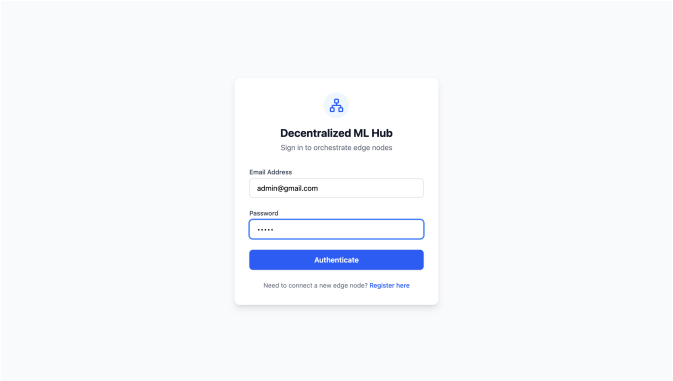

Fig. 1. Website Authentication: Secure Login and Registration portal into the server website.

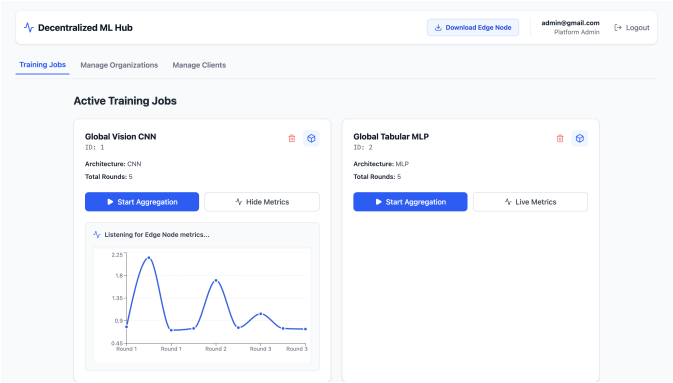

Fig. 2. Admin Dashboard: Displaying live telemetry graphs and global loss metrics over successive training rounds.

The Admin interface ensures that administrators have complete visibility into which nodes are contributing valuable weights and how those contributions are reducing the global model's loss function.

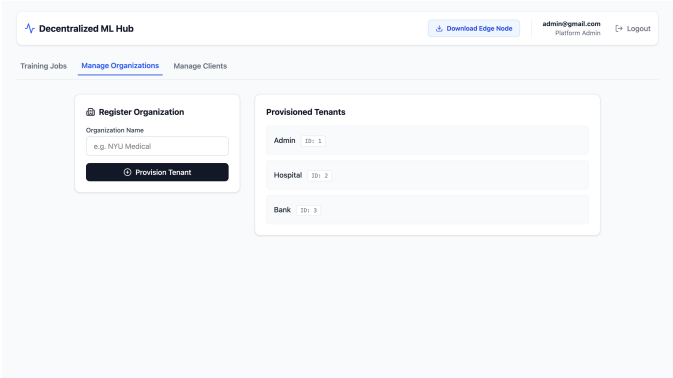

Fig. 3. Organization Management: Interface for creating and managing different institutional networks.

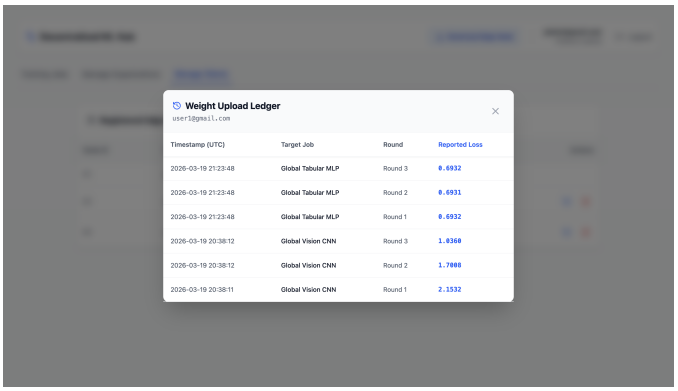

Fig. 4. Client Management: Granular view tracking individual edge node connections and the specific model weights they have submitted for aggregation.

IV. User Interface Design: Edge Client

The macOS edge client is a native desktop application designed for simplicity, abstracting away the complex machine learning operations so contributors can train models with a few clicks.

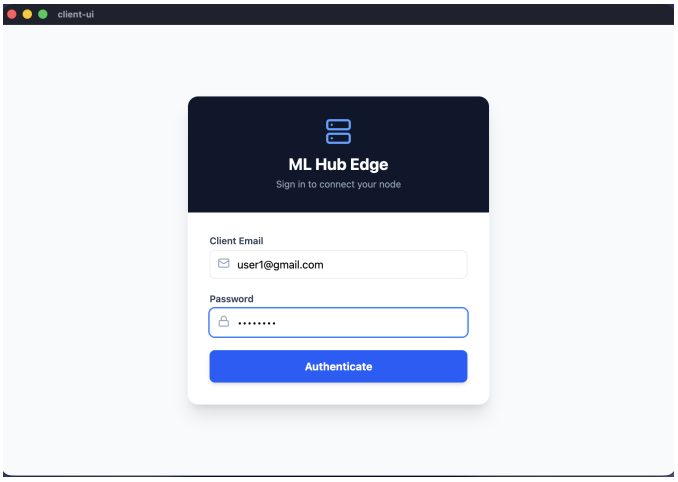

Fig. 5. Edge Node Authentication: Client-side Login and Registration to securely connect to the Hub.

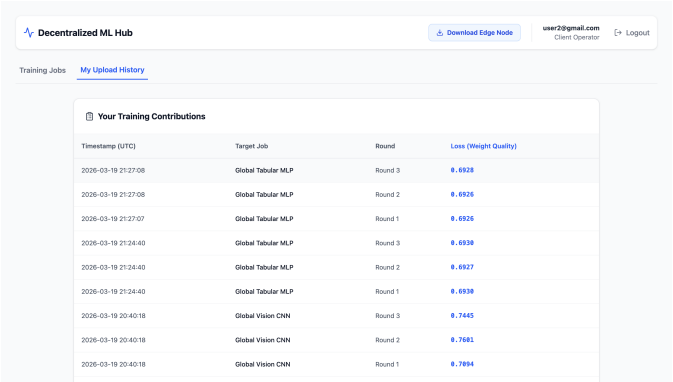

Fig. 6. Client Dashboard: Displaying assigned Training Jobs and the status of the Global Model.

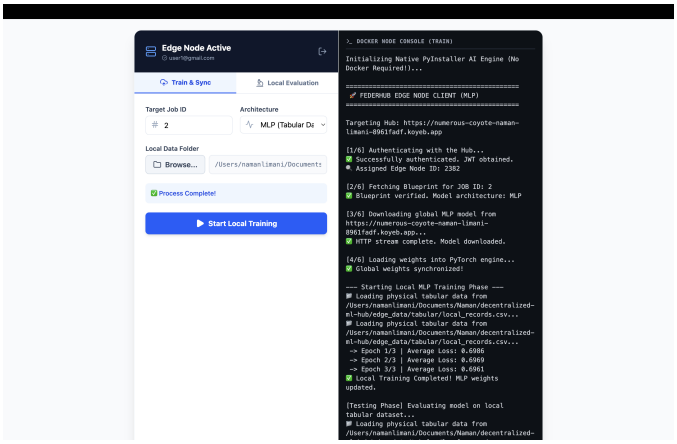

Fig. 7. Local Training Interface: Users select their private data folder, and the UI streams the native PyTorch stdout logs as the model trains locally.

By splitting the Training and Testing phases, the client UI provides transparency, ensuring the edge node verifies the integrity of its learning before transmitting updates to the central server.

V. Challenges & Solutions

Developing a decentralized platform introduced several complex engineering hurdles:

- **Cloud Firewalls & gRPC:** Initial attempts to stream model chunks via gRPC (port 50051) were blocked by strict cloud PaaS firewalls. This was solved by refactoring the PyTorch edge pipelines to utilize standard HTTP REST APIs over port 443, effectively bypassing the firewall.
- **Mac Security Sandboxes:** Early iterations relied on local Docker containers. However, macOS security sandboxes strip path variables from GUI apps, causing ‘ENOENT‘ errors. The solution was abandoning Docker entirely for native execution.
- **Binary Freezing & Electron Packaging:** Compiling the PyTorch engine resulted in a massive 1.5GB standalone binary. Electron Builder initially failed to package this

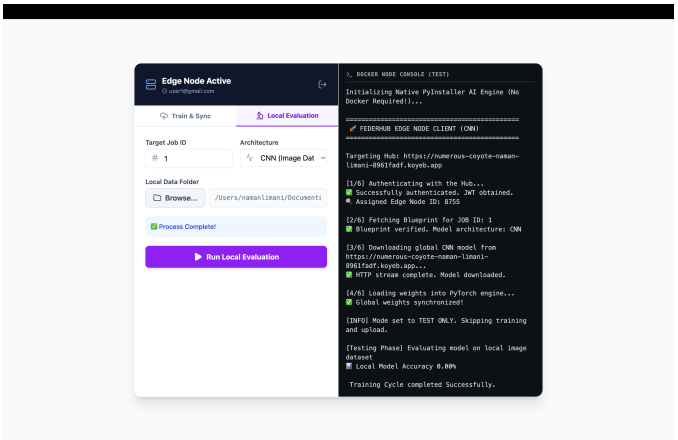

Fig. 8. Local Testing Interface: Evaluates the locally trained model against a holdout dataset to verify accuracy before uploading weights.

due to infinite recursion loops (‘ENAMETOOLONG‘). This was resolved by hyper-specifically targeting the compiled directory using the ‘extraResources‘ directive in the Electron configuration.

VI. Limitations & Workarounds

A current limitation of the platform exists within the cloud deployment tier. The central API is hosted on Koyeb’s free tier. By design, if the platform does not receive network traffic for a sustained period, the Koyeb instance enters a “sleep” state to conserve resources.

Consequently, if an edge node attempts to download a model while the server is asleep, the connection will time out.

Workaround: Currently, an administrator must manually log into the Koyeb console and click “Redeploy” to wake the server.

Automated Solution: To surpass this in the future without upgrading to a paid tier, a cron job or an external ping service (such as UptimeRobot) can be configured to send an HTTP GET request to the server every 10 minutes, preventing the instance from sleeping.

VII. Future Scope

The foundation of the Decentralized ML Hub allows for significant future expansion:

1) **Cross-Platform Support:** While the edge engine is currently compiled for macOS (Apple Silicon), PyInstaller can be utilized on Windows and Linux build machines to generate ‘.exe‘ and ‘.elf‘ executables, broadening the network.
2) **Advanced Aggregation Algorithms:** Integrating advanced algorithms like FedProx to handle statistical heterogeneity (non-IID data) across edge clients.
3) **Differential Privacy:** Injecting controlled noise into the local weights before transmission to guarantee mathematical privacy against model inversion attacks.

VIII. Conclusion

The Decentralized ML Hub successfully demonstrates that privacy-preserving AI training can be securely deployed to consumer hardware. By wrapping complex native machine learning execution within an intuitive desktop application and utilizing RESTful cloud communication, the platform eliminates the traditional friction of Docker and terminal interfaces, clearing the path for real-world federated learning applications in sensitive industries.